Cyber Shujaa

Security Analyst

ASSIGNMENT REPORT

THM - WiFi Hacking 101

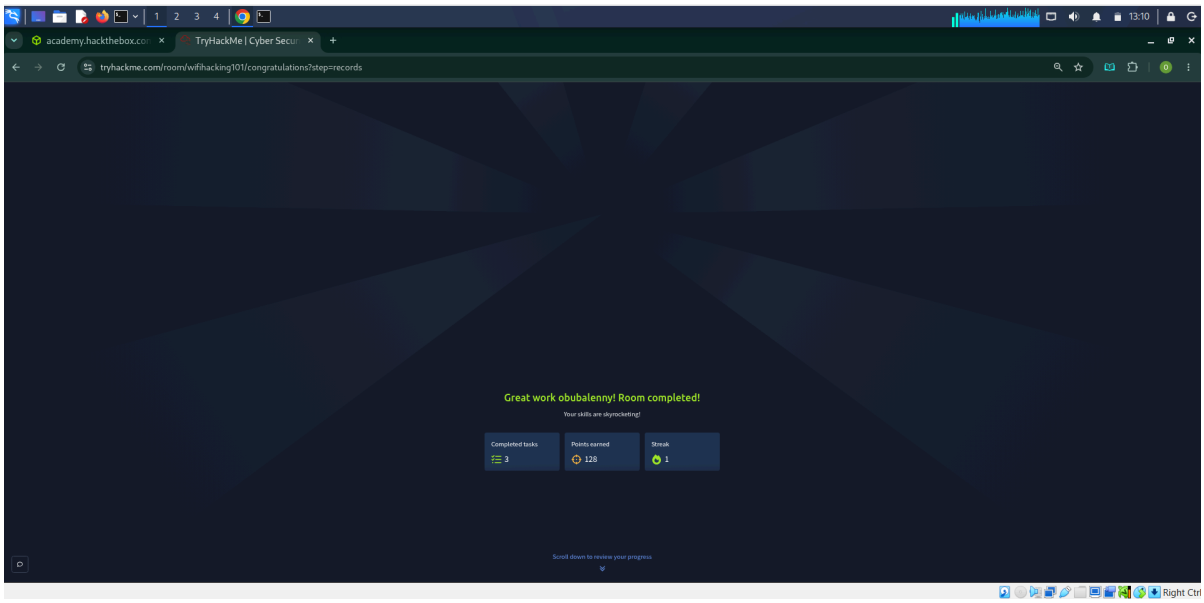
Full name	*Lenson Obuba*
Admission number	*cs-sa13-26017*
Email	*obubalenny@gmail.com*
Week and assignment	*Week 9, Assignment 1: THM - WiFi Hacking 101*
Platform	*TryHackMe*
Date submitted	*06/11/2026*

1. Module completion

Shareable link

https://tryhackme.com/room/wifihacking101/congratulations?step=room-completion-share

Completion screenshot



2. Introduction

This module covers how WPA/WPA2-Personal Wi-Fi networks authenticate clients, and how an attacker can capture the handshake exchanged during that process and attempt to recover the pre-shared key offline. As a SOC analyst, understanding this attack path matters because wireless networks are often the softest entry point into an otherwise well-defended environment; a weak passphrase on a guest or staff Wi-Fi network can undo a lot of other controls. Going into this room, I expected to get hands-on with the actual tools (airmon-ng/aircrack-ng style tooling).

3. The work

3.1 The Basics - An Intro to WPA

What this section covered

This section laid out the key vocabulary for wireless attacks which are SSID, ESSID, BSSID, WPA2-PSK vs WPA2-EAP, and RADIUS and explained that WPA/WPA2's security rests on the 4-way handshake, which lets a client and access point each prove they know the key without ever sending it over the air. I took away that WPA2-Personal networks are vulnerable to offline brute-force/dictionary attacks because the key is derived from the password plus the ESSID (used as a salt), meaning an attacker who captures the handshake can try passwords against it without touching the live network again, which is why password length and strength matter so much more here than on WPA2-EAP, which authenticates differently via RADIUS and isn't vulnerable to this same attack.

Question 1

What type of attack on the encryption can you perform on WPA(2) personal?

Answer

Brute force

Explanation: We can perform a brute force on WPA2 personal. It doesn't always work, but sometimes it's helpful to crack common passwords.

Question 2

Can this method be used to attack WPA2-EAP handshakes? (Yea/Nay)

Answer

Nay

Question 3

What is the three-letter abbreviation for the pre-shared key used in Wi-Fi security?

Answer

PSK

Explanation: PSK (Pre-Shared Key) is a WiFi security mechanism that uses the same passphrase to connect.

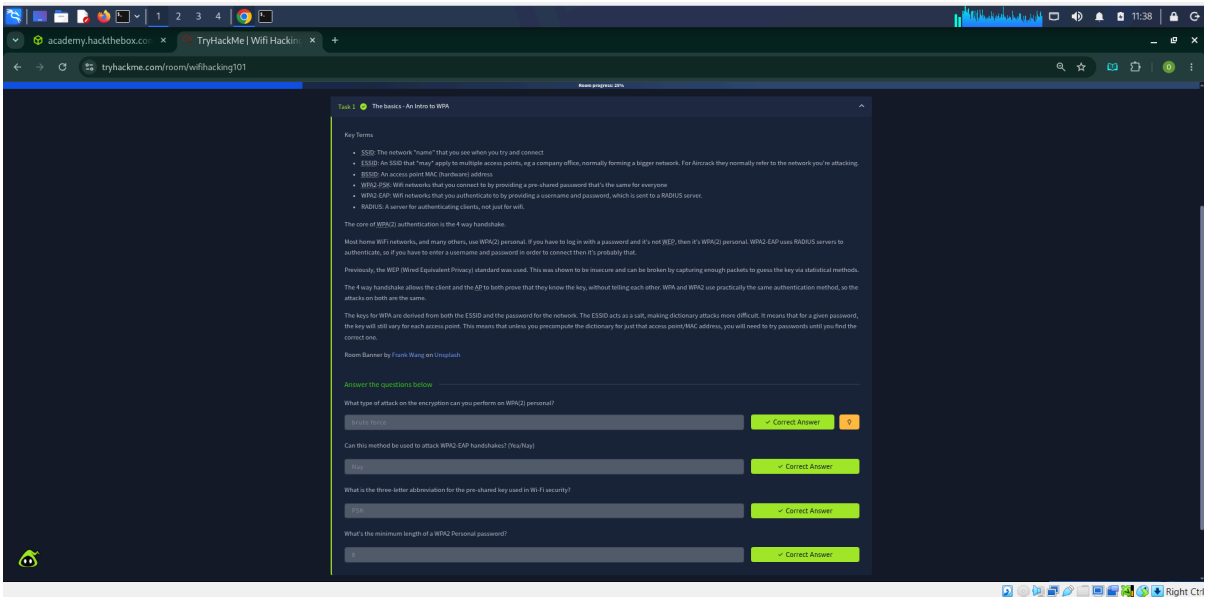
Question 4

What's the minimum length of a WPA2 Personal password?

Answer

8 (characters)

Explanation: WPA2 (Wifi Protected Access 2) has a minimum of 8 characters to secure the connection; however, hackers still can crack password if password is Weak and repetitive numbers and alphabets .



3.2 You're being watched - Capturing packets to attack

What this section covered

This section introduced the aircrack-ng suite (specifically airmon-ng, airodump-ng, and aircrack-ng) and the syntax for enabling monitor mode, killing interfering processes, and configuring airodump-ng to target a specific BSSID, channel, and output file. It explained that monitor mode gives raw access to 802.11 frames in the air (rather than only frames addressed to your own device, which is all "managed mode" exposes), which is what lets you passively capture the 4-way handshake, and that deauthentication (via injection mode) speeds this up by forcing a client to reconnect rather than waiting for one to do so naturally.

Question 1

How do you put the interface "wlan0" into monitor mode with Aircrack tools? (Full command)

Answer

airmon-ng start wlan0

Explanation - This switches the wlan0 interface from managed mode into monitor mode. Managed mode only lets the card see traffic addressed to it; monitor mode lets it capture all 802.11 frames in range, which is necessary to see the handshake between another client and the AP.

Question 2

What is the new interface name likely to be after you enable monitor mode?

Answer

wlan0mon

Explanation - airmon-ng typically renames the interface by appending mon to signal it's now in monitor mode, so scripts and commands downstream know which interface to target.

Question 3

What do you do if other processes are currently trying to use that network adapter?

Answer

airmon-ng check kill

Explanation - Processes like NetworkManager or wpa_supplicant actively manage the wireless card and will keep resetting it back to managed mode or interfering with the capture. Killing them first prevents monitor mode from silently failing or getting reverted mid-capture.

Question 4

What tool from the aircrack-ng suite is used to create a capture?

Answer

airodump-ng

Explanation - This is the packet-capture tool in the suite; it listens on the monitor-mode interface and writes the raw 802.11 traffic (including the handshake, once captured) to a file for later analysis/cracking.

Question 5

What flag do you use to set the BSSID to monitor?

Answer

--bssid

Explanation - This restricts the capture to one specific access point (identified by its MAC address) instead of grabbing all nearby Wi-Fi traffic, which keeps the capture clean and focused on the target network.

Question 6

and to set the channel?

Answer

--channel

Explanation - Wi-Fi networks operate on specific channels, and a card can only listen to one at a time. Locking to the target's channel avoids missing packets while the card hops around scanning other channels.

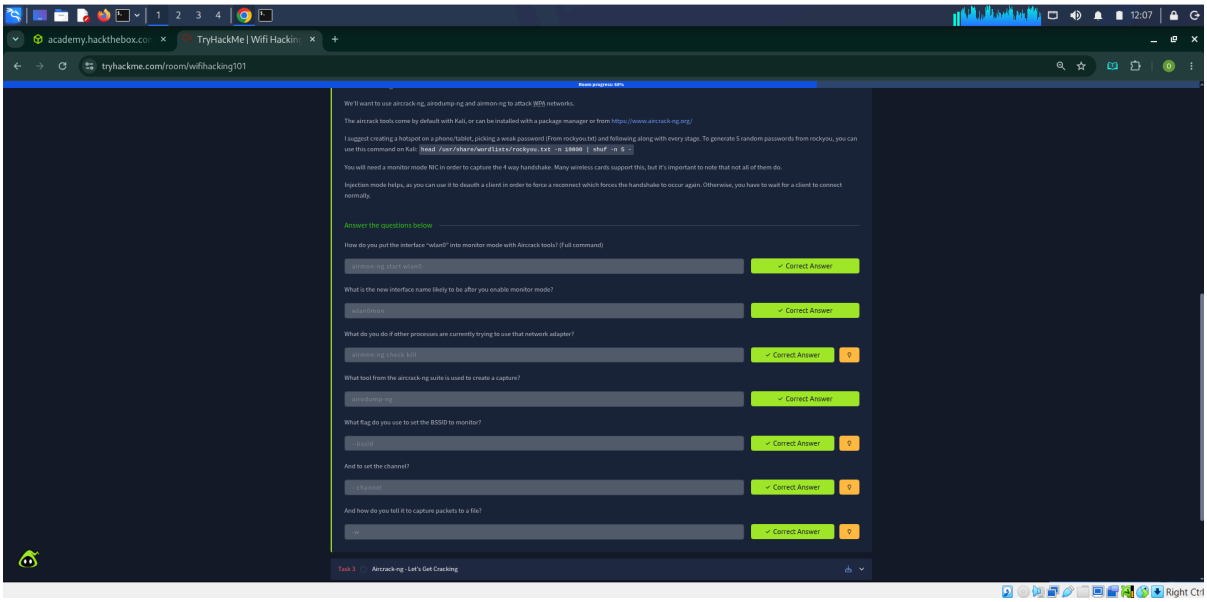
Question 7

And how do you tell it to capture packets to a file?

Answer

-w

Explanation - This tells airodump-ng to save the captured packets to a file on disk (rather than just displaying them live), which is what you need afterward to actually crack the handshake offline with aircrack-ng.



3.3 Aircrack-ng: Let's get cracking

What this section covered

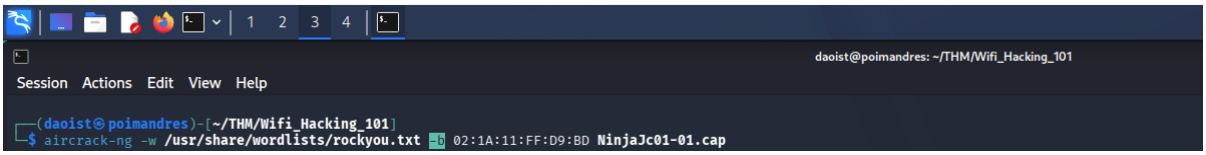
This section put the earlier theory into practice by cracking a WPA2 handshake capture offline using aircrack-ng and the rockyou wordlist, and introduced the alternative workflow of converting a capture into a hash file for GPU-accelerated cracking with hashcat.

Question 1

What flag do we use to specify a BSSID to attack?

Answer

-b

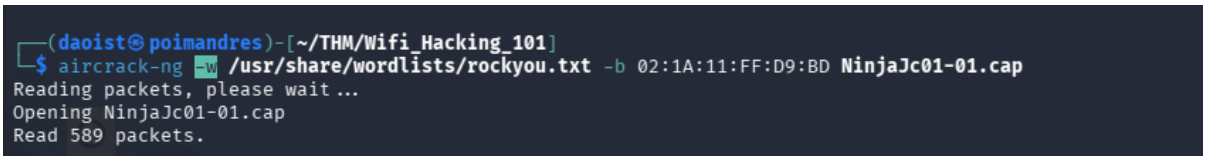


Question 2

What flag do we use to specify a wordlist?

Answer

-w



Question 3

How do we create a HCCAPX in order to use hashcat to crack the password?

Answer

-j

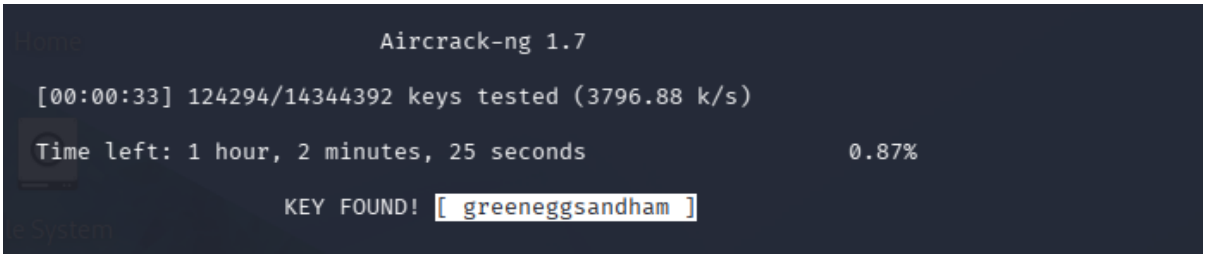
Explanation - -j: Hashcat doesn't crack .cap capture files directly , it needs the handshake data repackaged into a specific hash format it can iterate against. The -j flag tells aircrack-ng to parse the captured handshake from your .cap file and write out that data as a .hccapx file instead of running its own crack attempt. Lowercase -j specifically targets the newer .hccapx format (hashcat 3.6+), which is why it's the right flag for this question, uppercase -J would give you the older, now largely obsolete .hccap format instead. Once converted, you'd feed that .hccapx file to hashcat (mode 2500) to crack it with GPU acceleration rather than aircrack-ng's CPU-only engine.

Question 4

Using the rockyou wordlist, crack the password in the attached capture. What's the password?

Answer

greeneggsandham



Question 5

Where is password cracking likely to be fastest, CPU or GPU?

Answer

GPU

How you got there

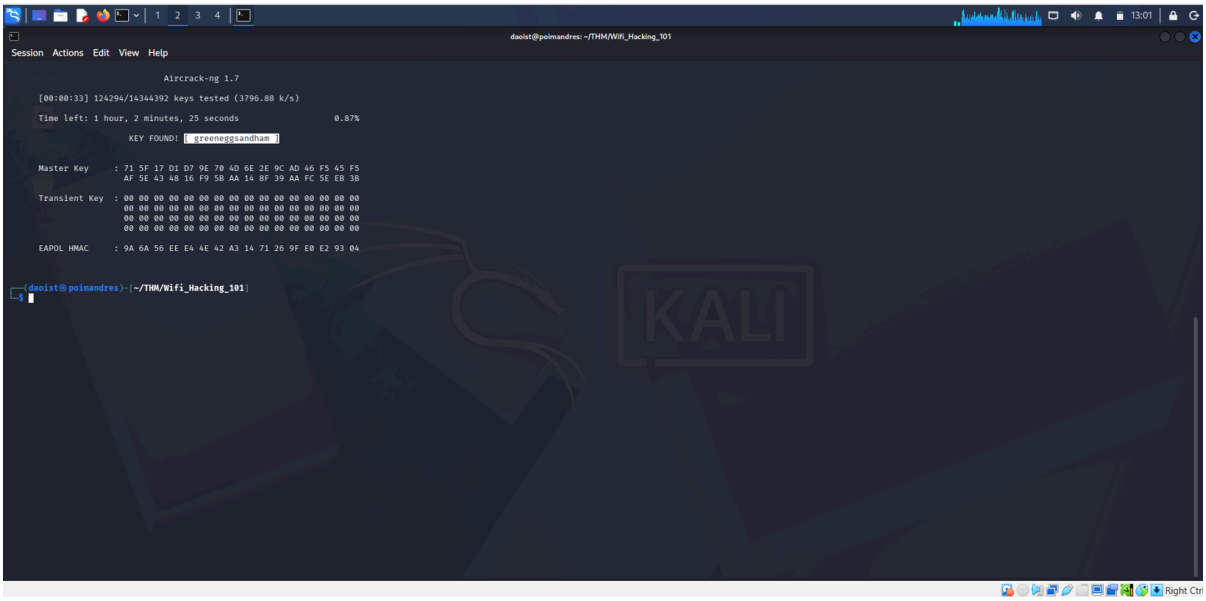
I downloaded the task capture file (NinjaJc01-01.cap, BSSID 02:1A:11:FF:D9:BD, ESSID "James Honor 8"), moved it into my working directory, and ran:

aircrack-ng -w /usr/share/wordlists/rockyou.txt -b 02:1A:11:FF:D9:BD NinjaJc01-01.cap

Aircrack-ng read the 589 packets in the capture, identified one potential target matching the BSSID, and began testing candidate passwords from rockyou.txt at roughly 3,874 keys/second. It found a match after testing 127,006 of 14,344,392 total keys (0.89% through the list), recovering the password greeneggsandham.

Evidence

Shows the terminal output of the aircrack-ng run showing the command, packet/target summary, and the "KEY FOUND!" result



Confirms the password for ESSID "James Honor 8" was successfully recovered via dictionary attack against the captured handshake.

4. Challenges and how you resolved them

The main obstacle in this room was hardware, not concepts. I don't have a monitor-mode-capable wireless adapter, my laptop's built-in Wi-Fi chipset doesn't support it, and VirtualBox virtualizes Wi-Fi as a standard NIC rather than exposing raw radio control to the guest VM, so even a capable built-in card wouldn't have helped without a USB adapter passed through directly. This meant I couldn't complete the live capture workflow in Task 2 (enabling monitor mode, running airodump-ng against a real access point, deauthenticating a client to force a handshake). Rather than skip that section, I worked through the aircrack-ng documentation and the room's own material to answer the syntax questions correctly.

5. Conclusion

Going in, I understood WPA2 as "password-protected Wi-Fi" but couldn't have explained what makes it secure or breakable. Now I can: the 4-way handshake lets the AP and client each prove they know the key without transmitting it, which is why an attacker has to capture that exchange and crack it offline rather than reading the password off the wire. I also understand why password length is the real defense, the ESSID salts the key derivation, so there's no shortcut like a precomputed table across networks, meaning a long, non-dictionary passphrase is what resists this attack, not complexity rules. "Use a strong password" used to be generic advice; now I can explain that an 8-character common word falls in seconds against a 14-million-entry wordlist, while a long passphrase makes the same offline attack impractical even with GPU acceleration.

What I learnt the most was how little of this happens on the live network, once the handshake is captured, cracking is entirely offline and undetectable, which reframes WPA2's weakness as a password

problem rather than a protocol flaw. Next time I'd set up a monitor-mode adapter before starting, so I could run the full attack chain myself rather than reasoning through part of it from documentation.